

EXAMEN PARCIAL - BASE DE DATOS II

Clases 1-7 | Multiple Choice

GRUPO A

PREGUNTA 1A - Relaciones M:N Complejas (SQL)

Nivel: Intermedio-Avanzado

Una empresa de e-commerce necesita modelar un sistema de pedidos, productos y descuentos complejos.

Requisitos:

Un producto puede tener multiples variantes. Los descuentos pueden ser por cantidad, codigo o cliente VIP. Cada item de pedido puede aplicar multiples descuentos acumulativos. Se debe registrar historial de cambios de precio. Pedidos tienen estados: pendiente, procesado, enviado, entregado.

Cual opcion define correctamente la tabla intermedia que gestiona la relacion entre items de pedido y descuentos, con contexto de calculo y auditoria?

A)

```
CREATE TABLE item_descuento (  
  id_item_descuento INT PRIMARY KEY AUTO_INCREMENT,  
  id_item_pedido INT NOT NULL,  
  id_descuento INT NOT NULL,  
  monto_descuento DECIMAL(10,2) NOT NULL,  
  fecha_aplicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_item_pedido) REFERENCES items_pedido(id_item),  
  FOREIGN KEY (id_descuento) REFERENCES descuentos(id_descuento),  
  UNIQUE KEY (id_item_pedido, id_descuento)  
);
```

B)

```
CREATE TABLE item_descuento (  
  id_item_pedido INT NOT NULL,  
  id_descuento INT NOT NULL,  
  secuencia INT NOT NULL,  
  monto_base DECIMAL(10,2) GENERATED ALWAYS AS (  
    SELECT valor_descuento  
    FROM descuentos d  
    WHERE d.id_descuento = id_descuento  
  ) STORED,  
  fecha_aplicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id_item_pedido, id_descuento, secuencia),  
  FOREIGN KEY (id_item_pedido) REFERENCES items_pedido(id_item),  
  FOREIGN KEY (id_descuento) REFERENCES descuentos(id_descuento)  
);
```

C)

```
CREATE TABLE item_descuento (  
    id_item_descuento      INT PRIMARY KEY AUTO_INCREMENT,  
    id_item_pedido         INT NOT NULL,  
    id_descuento           INT NOT NULL,  
    secuencia_aplicacion   INT NOT NULL,  
    valor_descuento_snapshot DECIMAL(10,2) NOT NULL,  
    monto_calculado        DECIMAL(10,2) NOT NULL,  
    estado_aplicacion      ENUM('pendiente','aplicado','revertido')  
                           NOT NULL DEFAULT 'pendiente',  
    fecha_registro         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    fecha_aplicacion       TIMESTAMP NULL,  
    usuario_aplicacion     VARCHAR(100),  
    FOREIGN KEY (id_item_pedido)  
        REFERENCES items_pedido(id_item) ON DELETE CASCADE,  
    FOREIGN KEY (id_descuento)  
        REFERENCES descuentos(id_descuento) ON DELETE RESTRICT,  
    UNIQUE KEY uk_item_desc_seq (id_item_pedido, secuencia_aplicacion),  
    INDEX      idx_estado_item (estado_aplicacion, id_item_pedido)  
);
```

D)

```
CREATE TABLE item_descuento (  
    id_item_pedido INT,  
    id_descuento   INT,  
    monto_final    DECIMAL(10,2),  
    PRIMARY KEY (id_item_pedido, id_descuento),  
    FOREIGN KEY (id_item_pedido) REFERENCES items_pedido(id_item),  
    FOREIGN KEY (id_descuento)   REFERENCES descuentos(id_descuento)  
);
```

E)

```
CREATE TABLE item_descuento (  
    id_item_descuento      INT PRIMARY KEY AUTO_INCREMENT,  
    id_item_pedido         INT NOT NULL,  
    id_descuento           INT NOT NULL,  
    secuencia_aplicacion   INT NOT NULL,  
    valor_descuento_snapshot DECIMAL(10,2) NOT NULL,  
    monto_calculado        DECIMAL(10,2) NOT NULL,  
    estado_aplicacion      ENUM('pendiente','aplicado','revertido')  
                           NOT NULL DEFAULT 'pendiente',  
    fecha_registro         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    fecha_aplicacion       TIMESTAMP NULL,  
    usuario_aplicacion     VARCHAR(100),  
    CHECK (monto_calculado >= valor_descuento_snapshot),  
    FOREIGN KEY (id_item_pedido)  
        REFERENCES items_pedido(id_item) ON DELETE CASCADE,  
    FOREIGN KEY (id_descuento)  
        REFERENCES descuentos(id_descuento) ON DELETE CASCADE,  
    UNIQUE KEY uk_item_descuento (id_item_pedido, id_descuento),  
    INDEX      idx_estado_item (estado_aplicacion, id_item_pedido)  
);
```

PREGUNTA 2A - Permisos y Niveles de Acceso (GRANT)

Nivel: Intermedio

Un equipo necesita configurar acceso a BD de produccion con tres perfiles: Analista de datos (solo lectura sobre toda la BD), Developer backend (DML unicamente en tablas 'pedidos' e 'items_pedido') y DBA (control total incluyendo gestion de usuarios).

Cual es la combinacion CORRECTA? (Hay 2 respuestas correctas que logran el mismo resultado con diferente granularidad. Selecciona ambas.)

A)

```
GRANT SELECT
  ON produccion.*
  TO 'analista'@'%';

GRANT SELECT, INSERT, UPDATE, DELETE
  ON produccion.pedidos
  TO 'developer'@'%';

GRANT SELECT, INSERT, UPDATE, DELETE
  ON produccion.items_pedido
  TO 'developer'@'%';

GRANT ALL PRIVILEGES
  ON *.*
  TO 'dba'@'localhost'
  WITH GRANT OPTION;
```

B)

```
GRANT SELECT
  ON produccion.*
  TO 'analista'@'%';

GRANT ALL PRIVILEGES
  ON produccion.*
  TO 'developer'@'%';

GRANT ALL PRIVILEGES
  ON *.*
  TO 'dba'@'localhost'
  WITH GRANT OPTION;
```

C)

```
GRANT SELECT, LOCK TABLES
  ON produccion.*
  TO 'analista'@'%';

GRANT SELECT, INSERT, UPDATE, DELETE
  ON produccion.pedidos, produccion.items_pedido
  TO 'developer'@'%';

GRANT SUPER, PROCESS, RELOAD
  ON *.*
  TO 'dba'@'localhost';
```

D)

```
GRANT SELECT
  ON produccion.*
  TO 'analista'@'%';

GRANT SELECT, INSERT, UPDATE, DELETE
  ON produccion.pedidos
  TO 'developer'@'%';

GRANT INSERT, UPDATE, DELETE
  ON produccion.items_pedido
  TO 'developer'@'%';

GRANT ALL PRIVILEGES
  ON *.*
  TO 'dba'@'localhost'
  WITH GRANT OPTION;
```

E)

```
GRANT SELECT
  ON produccion.*
  TO 'analista'@'%'
  WITH GRANT OPTION;

GRANT SELECT, INSERT, UPDATE, DELETE
  ON produccion.*
  TO 'developer'@'%';

GRANT ALL PRIVILEGES
  ON *.*
  TO 'dba'@'%';
```

F)

```
GRANT SELECT
  ON produccion.*
  TO 'analista'@'%';

GRANT CREATE TEMPORARY TABLES
  ON produccion.pedidos
  TO 'developer'@'%';

GRANT CREATE TEMPORARY TABLES
  ON produccion.items_pedido
  TO 'developer'@'%';

GRANT SUPER
  ON *.*
  TO 'dba'@'localhost';
```

PREGUNTA 3A - Modelado de Datos: JSON y XML

Nivel: Intermedio-Avanzado

Una plataforma de e-learning necesita representar un CURSO.

Requisitos:

El curso tiene metadatos: titulo, descripcion, y una lista de etiquetas (tags). Contiene multiples modulos ordenados, cada uno puede estar activo o inactivo. Cada modulo contiene multiples lecciones con tipo (video | lectura | evaluacion). Cada leccion puede contener sub_lecciones con identica estructura (recursividad). Cada leccion tiene una lista de recursos; cada recurso tiene: tipo, url, y duracion_segundos (presente solo si tipo es 'video', omitido en otros casos). Las etiquetas y los permisos de acceso son listas de strings.

Selecciona UNA respuesta correcta en JSON (opciones A-B-C) y UNA en XML (opciones D-E-F).

— Opciones en JSON —

A)

```
{
  "curso": {
    "metadatos": {
      "titulo": "Bases de Datos II",
      "descripcion": "Curso avanzado de SQL y NoSQL",
      "tags": ["sql", "nosql", "mongodb"],
      "permisos_acceso": ["estudiante", "docente"]
    },
    "modulos": [
      {
        "id_modulo": "M01",
        "nombre": "Introduccion a SQL",
        "orden": 1,
        "activo": true,
        "lecciones": [
          {
            "id_leccion": "L01",
            "titulo": "DDL: CREATE y ALTER",
            "orden": 1,
            "tipo": "video",
            "recursos": [
              { "tipo": "video", "url": "/media/ddl.mp4", "duracion_segundos": 1800 },
              { "tipo": "documento", "url": "/docs/ddl.pdf" }
            ],
            "sub_lecciones": [
              {
                "id_leccion": "L01-1",
                "titulo": "Tipos de datos en SQL",
                "orden": 1,
                "tipo": "lectura",
                "recursos": [
                  { "tipo": "documento", "url": "/docs/tipos.pdf" }
                ],
                "sub_lecciones": []
              }
            ]
          }
        ]
      }
    ]
  }
}
```

B)

```
{
  "curso": {
    "metadatos": {
      "titulo": "Bases de Datos II",
      "descripcion": "Curso avanzado de SQL y NoSQL",
      "tags": { "tag": ["sql", "nosql", "mongodb"] },
      "permisos_acceso": { "permiso": ["estudiante", "docente"] }
    },
    "modulos": {
      "modulo": {
        "id_modulo": "M01",
        "nombre": "Introduccion a SQL",
        "orden": 1,
        "activo": "true",
        "lecciones": {
          "leccion": {
            "id_leccion": "L01",
            "titulo": "DDL: CREATE y ALTER",
            "orden": 1,
            "tipo": "video",
            "recursos": {
              "recurso": [
                { "tipo": "video", "url": "/media/ddl.mp4",
                  "duracion_segundos": 1800 },
                { "tipo": "documento", "url": "/docs/ddl.pdf",
                  "duracion_segundos": null }
              ]
            }
          },
          "sub_lecciones": null
        }
      }
    }
  }
}
```

C)

```
{
  "curso": {
    "titulo": "Bases de Datos II",
    "descripcion": "Curso avanzado de SQL y NoSQL",
    "tags": ["sql", "nosql", "mongodb"],
    "permisos_acceso": ["estudiante", "docente"],
    "modulos": [
      {
        "id_modulo": "M01",
        "nombre": "Introduccion a SQL",
        "orden": 1,
        "activo": 1,
        "lecciones": [
          {
            "id_leccion": "L01",
            "titulo": "DDL: CREATE y ALTER",
            "orden": 1,
            "tipo": "video",
            "recursos": [
              { "tipo": "video", "url": "/media/ddl.mp4",
                "duracion_segundos": 1800 },
              { "tipo": "documento", "url": "/docs/ddl.pdf",
                "duracion_segundos": null }
            ]
          },
          "sub_lecciones": [
            {

```

```

    "id": "L01-1",
    "nombre": "Tipos de datos en SQL",
    "posicion": 1,
    "categoria": "lectura",
    "archivos": [
      { "formato": "documento", "ruta": "/docs/tipos.pdf" }
    ]
  }
]
}
}
```

— Opciones en XML —

D)

```
<?xml version="1.0" encoding="UTF-8"?>
<curso titulo="Bases de Datos II"
  descripcion="Curso avanzado de SQL y NoSQL"
  tags="sql,nosql,mongodb"
  permisos_acceso="estudiante,docente">
  <modulos>
    <modulo id_modulo="M01" nombre="Introduccion a SQL"
      orden="1" activo="true">
      <lecciones>
        <leccion id_leccion="L01" titulo="DDL: CREATE y ALTER"
          orden="1" tipo="video">
          <recursos>
            <recurso tipo="video" url="/media/ddl.mp4"
              duracion_segundos="1800"/>
            <recurso tipo="documento" url="/docs/ddl.pdf"/>
          </recursos>
          <sub_lecciones>
            <leccion id_leccion="L01-1" titulo="Tipos de datos"
              orden="1" tipo="lectura">
              <recursos>
                <recurso tipo="documento" url="/docs/tipos.pdf"/>
              </recursos>
            </sub_lecciones>
          </leccion>
        </sub_lecciones>
      </leccion>
    </lecciones>
  </modulo>
</modulos>
</curso>
```

E)

```
<?xml version="1.0" encoding="UTF-8"?>
<curso>
  <metadatos>
    <titulo>Bases de Datos II</titulo>
    <descripcion>Curso avanzado de SQL y NoSQL</descripcion>
    <tags>
      <tag>sql</tag>
      <tag>nosql</tag>
      <tag>mongodb</tag>
    </tags>
  </metadatos>
</curso>
```

```

    <permisos_acceso>
      <permiso>estudiante</permiso>
      <permiso>docente</permiso>
    </permisos_acceso>
  </metadatos>
  <modulos>
    <modulo id="M01" orden="1" activo="true">
      <nombre>Introduccion a SQL</nombre>
      <lecciones>
        <leccion id="L01" orden="1" tipo="video">
          <titulo>DDL: CREATE y ALTER</titulo>
          <recursos>
            <recurso tipo="video" url="/media/ddl.mp4">
              <duracion_segundos>1800</duracion_segundos>
            </recurso>
            <recurso tipo="documento" url="/docs/ddl.pdf"/>
          </recursos>
          <sub_lecciones>
            <leccion id="L01-1" orden="1" tipo="lectura">
              <titulo>Tipos de datos en SQL</titulo>
              <recursos>
                <recurso tipo="documento" url="/docs/tipos.pdf"/>
              </recursos>
            <sub_lecciones/>
          </leccion>
        </sub_lecciones>
      </leccion>
    </lecciones>
  </modulo>
</modulos>
</curso>

```

Motivo: Correcto: metadatos como elemento contenedor. Listas con elementos hermanos repetidos (tag, permiso). Atributos para datos cortos (id, orden, tipo, activo). Elementos para contenido textual. duracion_segundos como elemento hijo del recurso video, ausente en documento. sub_lecciones usa el mismo elemento leccion con identica estructura (recursividad real en XML).

F) INCORRECTA

```

<?xml version="1.0" encoding="UTF-8"?>
<curso xmlns:meta="http://elearning.com/meta"
  xmlns:cnt="http://elearning.com/content">
  <meta:metadatos>
    <meta:titulo>Bases de Datos II</meta:titulo>
    <meta:descripcion>Curso avanzado</meta:descripcion>
    <meta:tags>
      <meta:tag>sql</meta:tag>
      <meta:tag>nosql</meta:tag>
    </meta:tags>
  </meta:metadatos>
  <cnt:modulos>
    <cnt:modulo cnt:id="M01" cnt:orden="1">
      <cnt:nombre>Introduccion a SQL</cnt:nombre>
      <cnt:lecciones>
        <cnt:leccion cnt:id="L01" cnt:tipo="video">
          <cnt:titulo>DDL: CREATE y ALTER</cnt:titulo>
          <cnt:recursos>
            <cnt:recurso cnt:tipo="video"
              cnt:url="/media/ddl.mp4"
              cnt:duracion_segundos="1800"/>
            <cnt:recurso cnt:tipo="documento"
              cnt:url="/docs/ddl.pdf"
              cnt:duracion_segundos="0"/>
          </cnt:recursos>
        </cnt:leccion>
      </cnt:lecciones>
    </cnt:modulo>
  </cnt:modulos>
</curso>

```

PREGUNTA 4A - SQL Normalizado vs SQL con columnas JSON

Nivel: Intermedio-Avanzado

Una empresa necesita modelar su sistema de RRHH.

Requisitos:

Un empleado tiene datos fijos: nombre, email, fecha_ingreso, departamento y salario_base. Cada empleado puede tener multiples habilidades tecnicas con: nombre, nivel (junior/senior/experto) y años_experiencia. Se guarda historial de cambios salariales con: fecha, salario_anterior, salario_nuevo y motivo. Cada empleado tiene preferencias de notificacion (email, sms, push) que se leen y escriben siempre completas. Las habilidades y el historial son consultados frecuentemente con filtros y JOINS. Las preferencias de notificacion NUNCA se usan en JOINS ni filtros.

Hay dos enfoques validos: SQL totalmente normalizado y SQL hibrido (columnas fijas + JSON). Selecciona UNA opcion de SQL puro correcta (A/B/C) y UNA opcion hibrida correcta (D/E/F).

— Opciones SQL puro (normalizado) —

A)

```
CREATE TABLE empleados (  
    id_empleado    INT PRIMARY KEY AUTO_INCREMENT,  
    nombre         VARCHAR(200) NOT NULL,  
    email          VARCHAR(150) UNIQUE NOT NULL,  
    fecha_ingreso  DATE NOT NULL,  
    departamento   VARCHAR(100),  
    salario_base   DECIMAL(12,2) NOT NULL,  
    habilidades    VARCHAR(500),  
    historial      TEXT,  
    notificaciones VARCHAR(100)  
);
```

B)

```
CREATE TABLE empleados (  
    id_empleado    INT PRIMARY KEY AUTO_INCREMENT,  
    nombre         VARCHAR(200) NOT NULL,  
    email          VARCHAR(150) UNIQUE NOT NULL,  
    fecha_ingreso  DATE NOT NULL,  
    departamento   VARCHAR(100),  
    salario_base   DECIMAL(12,2) NOT NULL  
);  
  
CREATE TABLE habilidades (  
    id_habilidad   INT PRIMARY KEY AUTO_INCREMENT,  
    id_empleado    INT NOT NULL,  
    nombre         VARCHAR(100) NOT NULL,  
    nivel          VARCHAR(20),  
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)  
);  
  
CREATE TABLE historial_salarial (  
    id_historial   INT PRIMARY KEY AUTO_INCREMENT,  
    id_empleado    INT NOT NULL,  
    fecha          DATE NOT NULL,  
    salario_anterior DECIMAL(12,2),  
    salario_nuevo  DECIMAL(12,2) NOT NULL,  
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)  
);
```

C)

```
CREATE TABLE empleados (  
    id_empleado    INT PRIMARY KEY AUTO_INCREMENT,  
    nombre         VARCHAR(200) NOT NULL,  
    email          VARCHAR(150) UNIQUE NOT NULL,  
    fecha_ingreso  DATE NOT NULL,  
    departamento   VARCHAR(100),  
    salario_base   DECIMAL(12,2) NOT NULL  
);  
  
CREATE TABLE habilidades (  
    id_habilidad    INT PRIMARY KEY AUTO_INCREMENT,  
    id_empleado     INT NOT NULL,  
    nombre          VARCHAR(100) NOT NULL,  
    nivel           ENUM('junior','senior','experto') NOT NULL,  
    anios_experiencia TINYINT UNSIGNED NOT NULL DEFAULT 0,  
    FOREIGN KEY (id_empleado)  
        REFERENCES empleados(id_empleado) ON DELETE CASCADE,  
    INDEX idx_emp_nivel (id_empleado, nivel)  
);  
  
CREATE TABLE historial_salarial (  
    id_historial    INT PRIMARY KEY AUTO_INCREMENT,  
    id_empleado     INT NOT NULL,  
    fecha           DATE NOT NULL,  
    salario_anterior DECIMAL(12,2) NOT NULL,  
    salario_nuevo   DECIMAL(12,2) NOT NULL,  
    motivo          VARCHAR(300),  
    FOREIGN KEY (id_empleado)  
        REFERENCES empleados(id_empleado) ON DELETE CASCADE,  
    INDEX idx_emp_fecha (id_empleado, fecha)  
);  
  
CREATE TABLE notificaciones (  
    id_empleado INT PRIMARY KEY,  
    email       BOOLEAN NOT NULL DEFAULT TRUE,  
    sms         BOOLEAN NOT NULL DEFAULT FALSE,  
    push        BOOLEAN NOT NULL DEFAULT FALSE,  
    FOREIGN KEY (id_empleado)  
        REFERENCES empleados(id_empleado) ON DELETE CASCADE  
);
```

— Opciones SQL híbrido (columnas fijas + JSON) —

D)

```
CREATE TABLE empleados (  
    id_empleado    INT PRIMARY KEY AUTO_INCREMENT,  
    nombre         VARCHAR(200) NOT NULL,  
    email          VARCHAR(150) UNIQUE NOT NULL,  
    fecha_ingreso  DATE NOT NULL,  
    departamento   VARCHAR(100),  
    salario_base   DECIMAL(12,2) NOT NULL,  
    habilidades    JSON,  
    historial      JSON,  
    notificaciones JSON  
);
```

E)

```
CREATE TABLE empleados (  
  id_empleado    INT PRIMARY KEY AUTO_INCREMENT,  
  nombre         VARCHAR(200) NOT NULL,  
  email          VARCHAR(150) UNIQUE NOT NULL,  
  fecha_ingreso  DATE NOT NULL,  
  departamento   VARCHAR(100),  
  salario_base   DECIMAL(12,2) NOT NULL,  
  notificaciones JSON NOT NULL DEFAULT (  
    JSON_OBJECT('email', TRUE, 'sms', FALSE, 'push', FALSE)  
  )  
);  
  
CREATE TABLE habilidades (  
  id_habilidad    INT PRIMARY KEY AUTO_INCREMENT,  
  id_empleado     INT NOT NULL,  
  nombre          VARCHAR(100) NOT NULL,  
  nivel           ENUM('junior', 'senior', 'experto') NOT NULL,  
  anios_experiencia TINYINT UNSIGNED NOT NULL DEFAULT 0,  
  FOREIGN KEY (id_empleado)  
    REFERENCES empleados(id_empleado) ON DELETE CASCADE,  
  INDEX idx_emp_nivel (id_empleado, nivel)  
);  
  
CREATE TABLE historial_salarial (  
  id_historial    INT PRIMARY KEY AUTO_INCREMENT,  
  id_empleado     INT NOT NULL,  
  fecha           DATE NOT NULL,  
  salario_anterior DECIMAL(12,2) NOT NULL,  
  salario_nuevo   DECIMAL(12,2) NOT NULL,  
  motivo          VARCHAR(300),  
  FOREIGN KEY (id_empleado)  
    REFERENCES empleados(id_empleado) ON DELETE CASCADE,  
  INDEX idx_emp_fecha (id_empleado, fecha)  
);
```

F)

```
CREATE TABLE empleados (  
  id_empleado    INT PRIMARY KEY AUTO_INCREMENT,  
  nombre         VARCHAR(200) NOT NULL,  
  email          VARCHAR(150) UNIQUE NOT NULL,  
  fecha_ingreso  DATE NOT NULL,  
  departamento   VARCHAR(100),  
  salario_base   DECIMAL(12,2) NOT NULL,  
  notificaciones JSON  
);  
  
CREATE TABLE habilidades (  
  id_habilidad    INT PRIMARY KEY AUTO_INCREMENT,  
  id_empleado     INT NOT NULL,  
  datos           JSON NOT NULL,  
  FOREIGN KEY (id_empleado)  
    REFERENCES empleados(id_empleado) ON DELETE CASCADE  
);  
  
CREATE TABLE historial_salarial (  
  id_historial    INT PRIMARY KEY AUTO_INCREMENT,  
  id_empleado     INT NOT NULL,  
  datos           JSON NOT NULL,  
  FOREIGN KEY (id_empleado)  
    REFERENCES empleados(id_empleado) ON DELETE CASCADE  
);
```

PREGUNTA 5A - Arquitectura del Stack: Apache, PHP, MySQL y JSON

Nivel: Intermedio

Un equipo de desarrollo despliega una aplicacion web usando XAMPP. Al describir cada componente en la documentacion tecnica del proyecto, deben caracterizar con precision la naturaleza de cada tecnologia: si es un servidor, un servicio, un motor, un lenguaje interpretado, un lenguaje incrustado, un formato de intercambio, un tipo de dato nativo, o un protocolo. Identificar correctamente estos conceptos es critico para entender como interactuan los componentes en tiempo de ejecucion.

Selecciona la descripcion COMPLETAMENTE CORRECTA de todos los componentes:

A)

- Apache → Lenguaje del lado del servidor que compila codigo PHP y lo despacha al navegador como HTML estatico.
- PHP → Servidor web que escucha en el puerto 80 y enruta solicitudes HTTP hacia los scripts de base de datos.
- MySQL → Motor de base de datos relacional que se ejecuta como servicio (daemon) independiente del servidor web.
- JSON → Protocolo de comunicacion entre Apache y MySQL para transferir resultados de consultas SQL.
- HTML → Lenguaje interpretado que corre en el servidor y genera las respuestas dinamicas de la aplicacion.

B)

- Apache → Servidor web que actua como intermediario HTTP entre el navegador y el sistema de archivos del servidor; ejecuta directamente el codigo PHP en su proceso.
- PHP → Formato de intercambio de datos semiestructurados que permite serializar objetos PHP para almacenarlos en columnas VARCHAR de MySQL.
- MySQL → Servidor de aplicaciones que gestiona la logica de negocio y delega las consultas a Apache.
- JSON → Tipo de dato nativo en MySQL desde la version 5.7, validado al insertar, almacenado en formato binario y consultable con funciones como JSON_EXTRACT.
- HTML → Lenguaje de marcado interpretado por el navegador para renderizar la interfaz de usuario.

C)

- Apache → Servidor web (proceso daemon) que recibe peticiones HTTP, sirve archivos estaticos directamente y delega los archivos .php al interprete PHP para su ejecucion.
- PHP → Lenguaje de programacion interpretado del lado del servidor, habitualmente incrustado en HTML mediante las etiquetas <?php ... ?>, cuyo output es HTML puro enviado al navegador.
- MySQL → Motor de base de datos relacional (SGBD) que se ejecuta como servicio independiente; PHP se conecta a el mediante la extension mysqli o PDO.
- JSON → Formato de texto ligero para intercambio de datos (RFC 8259); en MySQL tambien existe como tipo de dato nativo (desde 5.7) con almacenamiento binario interno y funciones propias (JSON_EXTRACT, JSON_SET, etc.).
- HTML → Lenguaje de marcado interpretado por el navegador; no se ejecuta en el servidor sino en el cliente.

D)

- Apache → Servidor web (proceso daemon) que recibe peticiones HTTP y ejecuta directamente código PHP sin necesidad de un intérprete externo.
- PHP → Lenguaje compilado del lado del servidor que genera bytecode antes de la ejecución para mejorar la performance; el bytecode se envía al navegador.
- MySQL → Motor de base de datos relacional (SGBD) que se ejecuta como servicio independiente; PHP se conecta a él mediante la extensión mysqli o PDO.
- JSON → Formato de texto ligero para intercambio de datos (RFC 8259); en MySQL también existe como tipo de dato nativo (desde 5.7) con almacenamiento binario interno y funciones propias (JSON_EXTRACT, JSON_SET, etc.).
- HTML → Lenguaje de marcado interpretado por el navegador; no se ejecuta en el servidor sino en el cliente.

E)

- Apache → Servidor web (proceso daemon) que recibe peticiones HTTP, sirve archivos estáticos directamente y delega los archivos .php al intérprete PHP para su ejecución.
- PHP → Lenguaje de programación interpretado del lado del servidor, habitualmente incrustado en HTML mediante las etiquetas <?php ... ?>, cuyo output es HTML puro enviado al navegador.
- MySQL → Motor de base de datos relacional (SGBD) que se ejecuta como servicio independiente; PHP se conecta a él mediante la extensión mysqli o PDO.
- JSON → Formato de texto ligero para intercambio de datos (RFC 8259); en MySQL se almacena siempre como VARCHAR internamente, y las funciones JSON_EXTRACT parsean el texto en cada consulta sin cache binario.
- HTML → Lenguaje de marcado interpretado por el navegador; no se ejecuta en el servidor sino en el cliente.

F)

- Apache → Servidor web (proceso daemon) que recibe peticiones HTTP, sirve archivos estáticos directamente y delega los archivos .php al intérprete PHP para su ejecución.
- PHP → Lenguaje de programación interpretado del lado del servidor, habitualmente incrustado en HTML mediante las etiquetas <?php ... ?>, cuyo output es HTML puro enviado al navegador.
- MySQL → Motor de base de datos relacional (SGBD) que se ejecuta como servicio independiente; PHP se conecta a él mediante la extensión mysqli o PDO.
- JSON → Formato de texto ligero para intercambio de datos (RFC 8259); en MySQL también existe como tipo de dato nativo (desde 5.7) con almacenamiento binario interno y funciones propias (JSON_EXTRACT, JSON_SET, etc.).
- HTML → Lenguaje de programación interpretado del lado del servidor que genera respuestas dinámicas; el navegador lo recibe ya procesado como texto plano.